4 Protocolos de comunicación y ejemplos — 4.1 Protocolo CAN Simple

Traducción de la sección 4.1 del manual GIM6010-8 rev 2.2 (págs. 41–53 del original).

La interfaz de comunicación por defecto es CAN, con una velocidad máxima de 1 Mbps (se lee y se fija con `odrv0.can.config.baud_rate`); la velocidad de fábrica es 500 kbps. Tenga en cuenta que en las versiones de hardware antiguas (3.7 o anteriores) el USB y el CAN no son compatibles entre sí; consulte el punto 3) del apartado 3.1.8 para saber cómo pasar de USB a CAN.

El protocolo por defecto del driver es CAN Simple. Si el usuario ha cambiado al protocolo CANopen, puede volver a CAN Simple con:

```
odrv0.can.config.protocol = 1
```

Tenga en cuenta que, en las versiones de hardware 3.10 y posteriores, la interfaz de comunicación admite además RS485, pulso/dirección y PWM. Con el comando siguiente se asegura de que la interfaz es CAN:

```
odrv0.config.comm_intf_mux = 0
```

En el Motor Wizard también puede cambiarse la interfaz física y la versión del protocolo CAN desde los parámetros de comunicación.

4.1.1 Formato de las tramas del protocolo

La comunicación CAN usa el formato de trama estándar: trama de datos, ID de 11 bits y 8 bytes de datos, según la tabla siguiente (a la izquierda el MSB, a la derecha el LSB):

Campo	CAN ID (11 bits)		Datos (8 bytes)
Segmento	Bit10 ~ Bit5	Bit4 ~ Bit0	Byte0 ~ Byte7
Descripción	`node_id`	`cmd_id`	Datos de la comunicación

- **`node_id`:** ID único del motor en el bus; se lee y se fija en `odrivetool` con `odrv0.axis0.config.can.node_id`.
- **`cmd_id`:** código de comando; indica el tipo de mensaje del protocolo (véase el resto de esta sección).
- **Datos de la comunicación:** 8 bytes. Los parámetros que lleva cada mensaje se codifican como enteros o como números en coma flotante, con orden de bytes *little endian*; los números en coma flotante siguen la norma IEEE 754 (puede comprobarse la codificación en `https://www.h-schmidt.net/FloatConverter/IEEE754.html`).

Tomemos como ejemplo el mensaje `Set_Input_Pos` descrito en el apartado 4.1.2, suponiendo estos tres parámetros: `Input_Pos` = 3,14; `Vel_FF` = 1000 (equivale a 1 rev/s) y `Torque_FF` = 5000 (equivale a 5 N·m). El CMD ID de `Set_Input_Pos` es 0x00C y suponemos que el nodo del driver (`node_id`) es 0x05; entonces:

- El CAN ID de 11 bits es (0x05 << 5) + 0x0C = 0xAC.

- Según la descripción de `Set_Input_Pos` del apartado 4.1.2, `Input_Pos` ocupa 4 bytes a partir del byte 0 y se codifica como C3 F5 48 40 (el número 3,14 en IEEE 754 es 0x4048F5C3); `Vel_FF` ocupa 2 bytes a partir del byte 4 y se codifica como E8 03 (1000 = 0x03E8); `Torque_FF` ocupa 2 bytes a partir del byte 6 y se codifica como 88 13 (5000 = 0x1388). Los 8 bytes de datos quedan así:

Byte0	Byte1	Byte2	Byte3	Byte4	Byte5	Byte6	Byte7
C3	F5	48	40	E8	03	88	13

4.1.2 Mensajes

La tabla siguiente recoge todos los mensajes disponibles:

CMD ID	Nombre	Sentido	Parámetros
0x001	Heartbeat	motor → maestro	Axis_Error, Axis_State, Motor_Flag, Encoder_Flag, Controller_Flag, Traj_Done, Life
0x002	Estop	maestro → motor	—
0x003	Get_Error	motor → maestro	Error_Type
0x004	RxSdo	motor → maestro	—
0x005	TxSdo	motor → maestro	—
0x006	Set_Axis_Node_ID	maestro → motor	Axis_Node_ID
0x007	Set_Axis_State	maestro → motor	Axis_Requested_State
0x008	Mit_Control	maestro → motor	—
0x009	Get_Encoder_Estimates	motor → maestro	Pos_Estimate, Vel_Estimate
0x00A	Get_Encoder_Count	motor → maestro	Shadow_Count, Count_In_Cpr
0x00B	Set_Controller_Mode	maestro → motor	Control_Mode, Input_Mode
0x00C	Set_Input_Pos	maestro → motor	Input_Pos, Vel_FF, Torque_FF
0x00D	Set_Input_Vel	maestro → motor	Input_Vel, Torque_FF
0x00E	Set_Input_Torque	maestro → motor	Input_Torque
0x00F	Set_Limits	maestro → motor	Velocity_Limit, Current_Limit
0x010	Start_Anticogging	maestro → motor	—
0x011	Set_Traj_Vel_Limit	maestro → motor	Traj_Vel_Limit
0x012	Set_Traj_Accel_Limits	maestro → motor	Traj_Accel_Limit, Traj_Decel_Limit
0x013	Set_Traj_Inertia	maestro → motor	Traj_Inertia
0x014	Get_Iq	motor → maestro	Iq_Setpoint, Iq_Measured
0x016	Reboot	maestro → motor	—
0x017	Get_Bus_Voltage_Current	motor → maestro	Bus_Voltage, Bus_Current
0x018	Clear_Errors	maestro → motor	—
0x019	Set_Linear_Count	maestro → motor	Linear_Count
0x01A	Set_Pos_Gain	maestro → motor	Pos_Gain
0x01B	Set_Vel_Gains	maestro → motor	Vel_Gain, Vel_Integrator_Gain

0x01C	Get_Torques	motor → maestro	Torque_Setpoint, Torque
0x01D	Get_Powers	motor → maestro	Electrical_Power, Mechanical_Power
0x01E	Disable_Can	maestro → motor	—
0x01F	Save_Configuration	maestro → motor	—

La descripción detallada de todos los mensajes es la siguiente.

➢ Heartbeat

CMD ID: 0x001 (motor → maestro)

Formato del latido en las versiones de firmware 0.5.11 e anteriores:

Byte inicial	Nombre	Tipo	Descripción
0	Axis_Error	uint32	Código de error del driver (`odrv0.axis0.error`)
4	Axis_State	uint8	Estado del driver (`odrv0.axis0.current_state`)
5	Motor_Flag	uint8	1: error del motor (`odrv0.axis0.motor.error` distinto de 0) 0: motor correcto (`odrv0.axis0.motor.error` igual a 0)
6	Encoder_Flag	uint8	1: error del encoder (`odrv0.axis0.encoder.error` distinto de 0) 0: encoder correcto (`odrv0.axis0.encoder.error` igual a 0)
7	Controller_Flag	uint8	bit7: `odrv0.axis0.controller.trajectory_done`, es decir, si la trayectoria de posición ha terminado bit0: 1 = error de control (`odrv0.axis0.controller.error` distinto de 0); 0 = control correcto (igual a 0)

Formato del latido en las versiones de firmware 0.5.12 y posteriores:

Byte inicial	Nombre	Tipo	Descripción
0	Axis_Error	uint32	Código de error del driver (`odrv0.axis0.error`)
4	Axis_State	uint8	Estado del driver (`odrv0.axis0.current_state`)
5	Flags	uint8	bit0: bit de error del motor (si `odrv0.axis0.motor.error` es 0 o no) bit1: bit de error del encoder (`odrv0.axis0.encoder.error`) bit2: bit de error de control (`odrv0.axis0.controller.error`) bit7: `odrv0.axis0.controller.trajectory_done`, si la trayectoria de posición ha terminado
6	Reserved	uint8	Reservado
7	Life	uint8	Contador de vida del mensaje periódico: se incrementa en 1 con cada latido, con un rango de 0 a 255. Si el contador no es consecutivo, se han perdido latidos, es decir, la comunicación no es estable

Formato del latido en las versiones de firmware 0.5.13 y posteriores:

Byte inicial	Nombre	Tipo	Descripción
0	Axis_Error	uint32	Código de error del driver (`odrv0.axis0.error`)
4	Axis_State	uint8	Estado del driver (`odrv0.axis0.current_state`)
5	Flags	uint8	bit0: bit de error del motor (`odrv0.axis0.motor.error`) bit1: bit de error del encoder (`odrv0.axis0.encoder.error`) bit2: bit de error de control (`odrv0.axis0.controller.error`) bit3: bit de error del sistema (`odrv0.error`) bit7: `odrv0.axis0.controller.trajectory_done`, si la trayectoria de posición ha terminado
6	Reserved	uint8	Reservado
7	Life	uint8	Contador de vida del mensaje periódico: se incrementa en 1 con cada latido, con un rango de 0 a 255. Si no es consecutivo, se han perdido latidos y la comunicación no es estable

➢ Estop

CMD ID: 0x002 (maestro → motor). Sin parámetros ni datos.

Este comando provoca la parada de emergencia del motor y notifica el error `ESTOP_REQUESTED`.

➢ Get_Error

CMD ID: 0x003 (motor → maestro)

Entrada (maestro → motor):

Byte inicial	Nombre	Tipo	Descripción
0	Error_Type	uint8	0: obtener el error del motor 1: obtener el error del encoder 3: obtener el error de control 4: obtener el error del sistema

Salida (motor → maestro):

Byte inicial	Nombre	Tipo	Descripción
0	Error	uint32 / uint64	Los datos devueltos y su longitud dependen del `Error_Type` de entrada: 0: error del motor (`odrv0.axis0.motor.error`), uint64, 8 bytes 1: error del encoder (`odrv0.axis0.encoder.error`), uint32, 4 bytes 3: error de control (`odrv0.axis0.controller.error`), uint32, 4 bytes 4: error del sistema (`odrv0.error`), uint32, 4 bytes

➢ RxSdo

CMD ID: 0x004 (maestro → motor)

Entrada:

Byte inicial	Nombre	Tipo	Descripción
0	opcode	uint8	0: lectura 1: escritura
1	Endpoint_ID	uint16	Descargue el archivo JSON con los ID de todos los parámetros y funciones de interfaz: ➢ versión 0.5.13: `https://bl.cyberbeast.cn/actuator/endpoints_0.5.13.json` ➢ versión 0.5.14: `https://bl.cyberbeast.cn/actuator/endpoints_0.5.14.json`
3	Reservado	uint8	—
4	Value	uint8[4]	Depende del `Endpoint_ID`; véase la descripción del JSON anterior. Por ejemplo, si el `Endpoint_ID` corresponde a un valor `float` de lectura y escritura, estos 4 bytes son ese `float` codificado en IEEE 754, y con `opcode` = 1 se escribe en él

Salida (cuando el `opcode` anterior es 0):

Byte inicial	Nombre	Tipo	Descripción
0	opcode	uint8	Siempre 0
1	Endpoint_ID	uint16	Descargue el archivo JSON con los ID de todos los parámetros y funciones de interfaz: ➢ versión 0.5.13: `https://bl.cyberbeast.cn/actuator/endpoints_0.5.13.json` ➢ versión 0.5.14: `https://bl.cyberbeast.cn/actuator/endpoints_0.5.14.json`
3	Reservado	uint8	—
4	Value	uint8[4]	Depende del `Endpoint_ID`; véase la descripción del JSON anterior. Por ejemplo, si el `Endpoint_ID` corresponde a un valor `uint32` de lectura, estos 4 bytes son ese `uint32` en orden *little endian*

➢ TxSdo

CMD ID: 0x005 (motor → maestro)

Se usa igual que `RxSdo` con `opcode` = 1.

➢ Set_Axis_Node_ID

CMD ID: 0x006 (maestro → motor)

Byte inicial	Nombre	Tipo	Descripción
0	Axis_Node_ID	uint32	`odrv0.axis0.config.can.node_id` ID único del motor en el bus CAN

➢ Set_Axis_State

CMD ID: 0x007 (maestro → motor)

Byte inicial	Nombre	Tipo	Descripción

0	Axis_Requested_State	uint32	`odrv0.axis0.requested_state` ➢ 0: sin definir ➢ 1: reposo ➢ 3: calibración completa (motor + encoder) ➢ 4: calibración del motor ➢ 7: calibración del encoder ➢ 8: lazo cerrado

➢ Mit_Control

CMD ID: 0x008

Es una implementación equivalente al protocolo abierto de control de movimiento del MIT (`https://github.com/mit-biomimetics/Cheetah-Software`).

> **Importante:** en el control por USB, la posición, la velocidad y el par que se introducen se refieren al **lado del rotor**; en el control MIT por CAN, en cambio, se refieren al **lado del eje de salida**, para mantener la coherencia con el protocolo abierto del MIT.

✓ Maestro → motor

Bytes de la trama CAN	Significado	Descripción
BYTE0 BYTE1	**Posición:** 16 bits en total; BYTE0 son los 8 bits altos y BYTE1 los 8 bits bajos. Posición multivuelta del eje de salida, en radianes (RAD)	La posición real es de tipo `double` y hay que convertirla a un entero de 16 bits: `pos_int = (pos_double + 12.5) × 65535 / 25`

BYTE2
BYTE3
BYTE4
	Velocidad: 12 bits en total; BYTE2 son los 8 bits altos y BYTE3[7-4] (nibble alto) los 4 bits bajos. Es la velocidad angular del eje de salida, en RAD/s.
	Valor KP: 12 bits en total; BYTE3[3-0] (nibble bajo) son los 4 bits altos y BYTE4 los 8 bits bajos
		La velocidad real es de tipo `double` y hay que convertirla a un entero de 12 bits:
		`vel_int = (vel_double + 65) × 4095 / 130`
		El valor KP real es de tipo `double` y hay que convertirlo a un entero de 12 bits:
		`kp_int = kp_double × 4095 / 500`

BYTE5
BYTE6
BYTE7
	Valor KD: 12 bits en total; BYTE5 son los 8 bits altos y BYTE6[7-4] (nibble alto) los 4 bits bajos.
	Par: 12 bits en total; BYTE6[3-0] (nibble bajo) son los 4 bits altos y BYTE7 los 8 bits bajos. La unidad es el N·m
		El valor KD real es de tipo `double` y hay que convertirlo a un entero de 12 bits:
		`kd_int = kd_double × 4095 / 5`
		El par real es de tipo `double` y hay que convertirlo a un entero de 12 bits:
		`t_int = (t_double + 50) × 4095 / 100`
		La unidad de la constante de par es el N·m/A

✓ Motor → maestro

Bytes de la trama CAN	Significado	Descripción
BYTE0	node id	`node_id` del driver
BYTE1 BYTE2	**Posición:** 16 bits en total; BYTE1 son los 8 bits altos y BYTE2 los 8 bits bajos. Posición multivuelta del eje de salida, en radianes (RAD)	La posición real es de tipo `double` y se obtiene del entero de 16 bits así: `pos_double = pos_int × 25 / 65535 − 12.5`

BYTE3
BYTE4
BYTE5
	Velocidad: 12 bits en total; BYTE3 son los 8 bits altos y BYTE4[7-4] (nibble alto) los 4 bits bajos. Es la velocidad angular del eje de salida, en RAD/s.
	Par: 12 bits en total; BYTE4[3-0] (nibble bajo) son los 4 bits altos y BYTE5 los 8 bits bajos. La unidad es el N·m
		La velocidad real es de tipo `double` y se obtiene del entero de 12 bits así:
		`vel_double = vel_int × 130 / 4095 − 65`
		El par real es de tipo `double` y se obtiene del entero de 12 bits así:
		`t_double = t_int × 100 / 4095 − 50`
		La unidad de la constante de par es el N·m/A

➢ Get_Encoder_Estimates

CMD ID: 0x009 (motor → maestro)

Byte inicial	Nombre	Tipo	Unidad	Descripción
0	Pos_Estimate	float32	rev	`odrv0.axis0.encoder.pos_estimate` Posición actual del rotor del motor
4	Vel_Estimate	float32	rev/s	`odrv0.axis0.encoder.vel_estimate` Velocidad actual del rotor del motor

➢ Get_Encoder_Count

CMD ID: 0x00A (motor → maestro)

Byte inicial	Nombre	Tipo	Descripción
0	Shadow_Count	int32	`odrv0.axis0.encoder.shadow_count` Cuenta multivuelta del encoder
4	Count_In_Cpr	int32	`odrv0.axis0.encoder.count_in_cpr` Cuenta dentro de una vuelta del encoder

➢ Set_Controller_Mode

CMD ID: 0x00B (maestro → motor)

Byte inicial	Nombre	Tipo	Descripción
0	Control_Mode	uint32	`odrv0.axis0.controller.config.control_mode` — modo de control: 0: control de tensión 1: control de par 2: control de velocidad 3: control de posición
4	Input_Mode	uint32	`odrv0.axis0.controller.config.input_mode` — modo de entrada; indica de qué manera el valor introducido por el usuario gobierna el motor: 0: inactivo 1: control directo 2: rampa de velocidad 3: filtro de posición 5: curva trapezoidal 6: rampa de par 9: control de movimiento (MIT)

➢ Set_Input_Pos

CMD ID: 0x00C (maestro → motor)

Byte inicial	Nombre	Tipo	Unidad	Descripción
0	Input_Pos	float32	rev	`odrv0.axis0.controller.input_pos` Consigna del control de posición
4	Vel_FF	int16	0,001 rev/s	`odrv0.axis0.controller.input_vel` Velocidad de prealimentación del control de posición
6	Torque_FF	int16	0,001 N·m	`odrv0.axis0.controller.input_torque` Par de prealimentación del control de posición

➢ Set_Input_Vel

CMD ID: 0x00D (maestro → motor)

Byte inicial	Nombre	Tipo	Unidad	Descripción
0	Input_Vel	float32	rev/s	`odrv0.axis0.controller.input_vel` Consigna del control de velocidad
4	Torque_FF	float32	N·m	`odrv0.axis0.controller.input_torque` Par de prealimentación del control de velocidad

➢ Set_Input_Torque

CMD ID: 0x00E (maestro → motor)

Byte inicial	Nombre	Tipo	Unidad	Descripción
0	Input_Torque	float32	N·m	`odrv0.axis0.controller.input_torque` Consigna del control de par

➢ Set_Limits

CMD ID: 0x00F (maestro → motor)

Byte inicial	Nombre	Tipo	Unidad	Descripción
0	Velocity_Limit	float32	rev/s	`odrv0.axis0.controller.config.vel_limit` Límite global de velocidad
4	Current_Limit	float32	A	`odrv0.axis0.motor.config.current_lim` Límite de corriente del motor

➢ Start_Anticogging

CMD ID: 0x010 (maestro → motor)

Lanza la calibración del rizado de par (*cogging*).

➢ Set_Traj_Vel_Limit

CMD ID: 0x011 (maestro → motor)

Byte inicial	Nombre	Tipo	Unidad	Descripción
0	Traj_Vel_Limit	float32	rev/s	`odrv0.axis0.trap_traj.config.vel_limit` Límite de velocidad del control de posición trapezoidal

➢ Set_Traj_Accel_Limits

CMD ID: 0x012 (maestro → motor)

Byte inicial	Nombre	Tipo	Unidad	Descripción
0	Traj_Accel_Limit	float32	rev/s²	`odrv0.axis0.trap_traj.config.accel_limit` Límite de aceleración del control de posición trapezoidal
4	Traj_Decel_Limit	float32	rev/s²	`odrv0.axis0.trap_traj.config.decel_limit` Límite de deceleración del control de posición trapezoidal

➢ Set_Traj_Inertia

CMD ID: 0x013 (maestro → motor)

Byte inicial	Nombre	Tipo	Unidad	Descripción
0	Traj_Inertia	float32	N·m/(rev/s²)	`odrv0.axis0.controller.config.inertia` Inercia

➢ Get_Iq

CMD ID: 0x014 (motor → maestro)

Byte inicial	Nombre	Tipo	Unidad	Descripción
0	Iq_Setpoint	float32	A	`odrv0.axis0.motor.current_control.Idq_setpoint` Consigna de corriente del eje Q
4	Iq_Measured	float32	A	`odrv0.axis0.motor.current_control.Iq_measured` Corriente del eje Q realmente medida

➢ Reboot

CMD ID: 0x016 (maestro → motor)

➢ Get_Bus_Voltage_Current

CMD ID: 0x017 (motor → maestro)

Byte inicial	Nombre	Tipo	Unidad	Descripción
0	Bus_Voltage	float32	V	`odrv0.vbus_voltage` Tensión del bus
4	Bus_Current	float32	A	`odrv0.ibus` Corriente del bus

➢ Clear_Errors

CMD ID: 0x018 (maestro → motor)

Borra todos los errores y avisos.

➢ Set_Linear_Count

CMD ID: 0x019 (maestro → motor)

Fija la posición absoluta del encoder.

Byte inicial	Nombre	Tipo	Descripción
0	Linear_Count	int32	`odrv0.axis0.encoder.set_linear_count()` Fija la posición absoluta del encoder (valor de cuenta)

➢ Set_Pos_Gain

CMD ID: 0x01A (maestro → motor)

Byte inicial	Nombre	Tipo	Unidad	Descripción
0	Pos_Gain	float32	(rev/s)/rev	`odrv0.axis0.controller.config.pos_gain` Ganancia Kp del lazo de posición

➢ Set_Vel_Gains

CMD ID: 0x01B (maestro → motor)

Byte inicial	Nombre	Tipo	Unidad	Descripción
0	Vel_Gain	float32	N·m/(rev/s)	`odrv0.axis0.controller.config.vel_gain` Ganancia Kp del lazo de velocidad
4	Vel_Integrator_Gain	float32	N·m/rev	`odrv0.axis0.controller.config.vel_integrator_gain` Ganancia Ki del lazo de velocidad

➢ Get_Torques

CMD ID: 0x01C (motor → maestro)

Byte inicial	Nombre	Tipo	Unidad	Descripción
0	Torque_Setpoint	float32	N·m	`odrv0.axis0.controller.torque_setpoint` Consigna de par del lazo de corriente
4	Torque	float32	N·m	Par actual

➢ Get_Powers

CMD ID: 0x01D (motor → maestro)

Byte inicial	Nombre	Tipo	Descripción
0	Electrical_Power	float32	`odrv0.axis0.controller.electrical_power` Potencia eléctrica
4	Mechanical_Power	float32	`odrv0.axis0.controller.mechanical_power` Potencia mecánica

➢ Disable_Can

CMD ID: 0x01E (maestro → motor)

Deshabilita el CAN y reinicia el driver.

➢ Save_Configuration

CMD ID: 0x01F (maestro → motor)

Guarda la configuración actual, la aplica y reinicia.

4.1.3 El protocolo CAN en la práctica

4.1.3.1 Caso práctico: calibración tras dar tensión

La secuencia de mensajes CAN que hay que enviar es:

CAN ID	Tipo de trama	Datos de la trama	Descripción
0x007	Trama de datos	`04 00 00 00 00 00 00 00`	Mensaje: `Set_Axis_State` Parámetro: 4 Calibra el motor
0x007	Trama de datos	`07 00 00 00 00 00 00 00`	Mensaje: `Set_Axis_State` Parámetro: 7 Calibra el encoder

4.1.3.2 Caso práctico: control de velocidad

La secuencia de mensajes CAN que hay que enviar es:

CAN ID	Tipo de trama	Datos de la trama	Descripción
0x00B	Trama de datos	`02 00 00 00 02 00 00 00`	Mensaje: `Set_Controller_Mode` Parámetros: 2/2 Fija el modo de control en velocidad y el modo de entrada en rampa de velocidad
0x007	Trama de datos	`08 00 00 00 00 00 00 00`	Mensaje: `Set_Axis_State` Parámetro: 8 Entra en el estado de control en lazo cerrado
0x00D	Trama de datos	`00 00 20 41 00 00 00 00`	Mensaje: `Set_Input_Vel` Parámetros: 10/0 Fija la velocidad objetivo y el par de prealimentación: velocidad objetivo 10 (float 0x41200000) y par de prealimentación 0 (float 0x00000000)

4.1.3.3 Caso práctico: control de posición

La secuencia de mensajes CAN que hay que enviar es:

CAN ID	Tipo de trama	Datos de la trama	Descripción

0x00B	Trama de datos	`03 00 00 00 03 00 00 00`	Mensaje: `Set_Controller_Mode` Parámetros: 3/3 Fija el modo de control en posición y el modo de entrada en filtro de posición
0x007	Trama de datos	`08 00 00 00 00 00 00 00`	Mensaje: `Set_Axis_State` Parámetro: 8 Entra en el estado de control en lazo cerrado
0x00C	Trama de datos	`CD CC 0C 40 00 00 00 00`	Mensaje: `Set_Input_Pos` Parámetros: 2,2/0/0 Fija la posición objetivo y las prealimentaciones de velocidad y de par: posición objetivo 2,2 (float 0x400CCCCD), prealimentaciones de par y de velocidad a 0

Nota: el CAN ID de la tabla es el `cmd_id`; el ID que viaja realmente por el bus es `(node_id << 5) + cmd_id`. Los ejemplos suponen `node_id` = 0.

4.1.4 Compatibilidad con CANopen

Si los *node ID* se reparten adecuadamente, este protocolo puede convivir con CANopen. La tabla siguiente recoge las combinaciones válidas de *node ID* de CANopen y de este protocolo:

Node ID de CANopen	Node ID de este protocolo
32 … 127	0x10, 0x18, 0x20, 0x28
64 … 127	0x10, 0x11, 0x18, 0x19, 0x20, 0x21, 0x28, 0x29
96 … 127	0x10, 0x11, 0x12, 0x18, 0x19, 0x1A, 0x20, 0x21, 0x22, 0x28, 0x29, 0x2A

4.1.5 Mensajes periódicos

El usuario puede configurar el motor para que envíe mensajes periódicamente al equipo de control, sin que este tenga que pedirlos. Los mensajes periódicos se activan y desactivan mediante una serie de parámetros bajo `odrv0.axis0.config.can` (el valor 0 los desactiva; cualquier otro valor es el periodo en ms), como muestra la tabla:

Mensaje	Parámetro en odrivetool	Valor por defecto
Heartbeat	`odrv0.axis0.config.can.heartbeat_rate_ms`	100
Get_Encoder_Estimates	`odrv0.axis0.config.can.encoder_rate_ms`	10
Get_Motor_Error	`odrv0.axis0.config.can.motor_error_rate_ms`	0
Get_Encoder_Error	`odrv0.axis0.config.can.encoder_error_rate_ms`	0
Get_Controller_Error	`odrv0.axis0.config.can.controller_error_rate_ms`	0
Get_Sensorless_Error	`odrv0.axis0.config.can.sensorless_error_rate_ms`	0
Get_Encoder_Count	`odrv0.axis0.config.can.encoder_count_rate_ms`	0
Get_Iq	`odrv0.axis0.config.can.iq_rate_ms`	0
Get_Sensorless_Estimates	`odrv0.axis0.config.can.sensorless_rate_ms`	0

Get_Bus_Voltage_Current	`odrv0.axis0.config.can.bus_vi_rate_ms`	0

De fábrica, los dos primeros mensajes periódicos vienen activados, por lo que al observar el bus CAN se ven ambos mensajes emitiéndose con el periodo configurado. Pueden desactivarse con:

```
odrv0.axis0.config.can.heartbeat_rate_ms = 0
odrv0.axis0.config.can.encoder_rate_ms = 0
```

El contenido detallado de cada mensaje está en el apartado 4.1.2.